Klir: An ESP32-Based Smart Toilet System with Automated Lid Control, Sensor-Triggered Flushing, and UV-C Disinfection

In partial fulfillment

For the course requirement in

Bachelor of Science in Information Technology

Prepared by:

Alvarez, James Carl V.
Carlos, Dennis
Lindog Maria Rollainne R.
Lopez, Justine M.
Los Baños, Christian Ivan A.

BSIT 3A

Dr. Alvin Jason A. Virata
Professor

January 2026

St. Dominic Complex, E. Aguinaldo Highway, Bacoor City, Cavite, Philippines 4102
0917 625 8222 • 0922 850 9200 • (046) 417 7322 • (046) 417 8161
www.stdominiccollege.edu.ph stdominiccollege @sdcapikemen

Table of Contents

Chapter 1 ..... **4**

Introduction **4**

1.1 Overview of IoT/IoE Project 4

1.2 Purpose/Objective of the Project 4

1.3 Project Scope and Limitation 4

1.4 Significance and Impact of IoT/IoE Project 5

1.5 Key Terms and Definitions 5

Chapter 2 ..... **6**

Project Background and Motivation **6**

2.1 Problem Statement 6

2.2 Existing Solutions and Gaps 6

Chapter 3 ..... **7**

System Design and Architecture **7**

3.1 System Overview 7

3.2 System Components 8

3.2.1 Hardware Components 8

3.2.2 Software and Cloud Infrastructure 8

3.3.1 Hardware-to-Cloud Communication 9

3.3.2 Cloud-to-Frontend Monitoring 10

3.3.3 Frontend-to-Hardware Control 10

Chapter 4 ..... **11**

Hardware and Software Implementation **11**

4.1 Hardware Components (e.g., sensors, microcontrollers) 11

4.2 Software/Platform Used 13

Chapter 5 ..... **16**

St. Dominic Complex, E. Aguinaldo Highway, Bacoor City, Cavite, Philippines 4102
0917 625 8222 • 0922 850 9200 • (046) 417 7322 • (046) 417 8161
www.stdominiccollege.edu.ph stdominiccollege @sdcapikemen

Project Development and Implementation Process..... 16

5.1 Project Planning and Design..... 16

5.2 Project Development Phases..... 17

5.3 Hardware Setup and Installation..... 17

5.4 Software Development and Configuration..... 18

5.5 System Testing and Validation..... 19

5.6 Deployment and Integration..... 20

Chapter 6..... 20

Results and Findings..... 20

6.1 System Performance..... 20

6.2 Challenges Faced and How They Were Solved..... 23

Chapter 7..... 24

Conclusion..... 24

7.1 Summary of the Project..... 24

7.2 Future Work and Enhancements (Recommendations)..... 25

Appendices..... 30

St. Dominic Complex, E. Aguinaldo Highway, Bacoor City, Cavite, Philippines 4102
0917 625 8222 • 0922 850 9200 • (046) 417 7322 • (046) 417 8161
www.stdominiccollege.edu.ph stdominiccollege @sdcapikemen

Chapter 1

Introduction

1.1 Overview of IoT/IoE Project

This IoT project, titled Toilet Smart Flush System with UV Light, is inspired by advanced Japanese toilet systems commonly found in hotels that feature touch sensors and automated capabilities. The prototype utilizes an ESP32 microcontroller (NodeMCU form factor) integrated with various hardware components to automate bathroom hygiene

1.2 Purpose/Objective of the Project

The primary purpose of this project is to provide a structured technical architecture for a localized smart toilet system. The objectives include:

- Defining the hardware control system and web dashboard interface.
- Implementing real time data streaming via MQTT and local database storage .
- Ensuring the system utilizes cloud infrastructure (Firebase) for remote monitoring.
- Developing a React Native mobile application that delivers real-time push notifications to maintenance personnel and enables task acknowledgment and completion tracking.

1.3 Project Scope and Limitation

In Scope:

- Automated flush, lid control, and UV-C disinfection systems.
- Web dashboard for real-time monitoring and hardware configuration
- Water usage analytics, reporting, and alert notifications.
- User authentication and role-based access control
- React Native mobile application for maintenance personnel featuring push notifications, cleaning task acknowledgment, task completion marking, and cleaning history viewing

- Small plastic toilet model used as the physical prototype for hardware demonstration

Limitations

- Integration with third-party smart home platforms.
- The mobile application is limited to the maintenance personnel role and does not provide administrative controls available on the web dashboard
- The physical prototype uses a small plastic toilet model for demonstration purposes; full-scale integration with standard plumbing fixtures is outside the current scope
- The mobile application requires an active internet connection and does not support offline functionality in this version

1.4 Significance and Impact of IoT/IoE Project

By integrating the ESP32 microcontroller with Firebase Firestore, this project provides a bidirectional communication framework capable of monitoring environmental data with a control latency of under 1.5 seconds (Cloud-Enabled IoT, 2026). Furthermore, the shift to a cloud-powered platform enables proactive operational oversight, allowing administrators to use data-driven insights to manage hygiene rather than relying on reactive manual checks (Satheesh Kumar et al., 2025).

1.5 Key Terms and Definitions

Term	Definition
ESP32	**A small and affordable microcontroller that has built-in Wi-Fi and Bluetooth. It is commonly used in IoT/IoE projects to connect sensors and devices to the internet.**

MQTT (Message Queuing Telemetry Transport)	**A lightweight communication protocol used in IoT systems. It allows devices to send and receive data efficiently over the internet, even with low bandwidth.**
UV-C	**A type of ultraviolet light used for killing bacteria, viruses, and other microorganisms. It is commonly used for disinfection and sanitation systems.**
Next.js	**A framework based on React that is used to build web applications. It helps developers create both the front-end interface and backend API routes in one project.**

Chapter 2

Project Background and Motivation

2.1 Problem Statement

When a standard toilet is flushed, it creates a 'toilet plume'—an invisible spray of aerosolized waste. Recent laser-visualization research demonstrated that these particles can travel at 6.6 feet per second, reaching the breathing height of an average adult (4.9 feet) in just 8 seconds (Crimaldi, 2022).

2.2 Existing Solutions and Gaps

While basic smart toilets exist, many localized setups fail to provide administrators with accessible data regarding water consumption and system health. Modern IoT infrastructure

relies heavily on scalable cloud integration to solve this. Utilizing cloud-based NoSQL databases allows for real-time visualization and analytics across multiple platforms without the bottleneck of local server maintenance (Usha et al., 2025)

Chapter 3

System Design and Architecture

3.1 System Overview

The proposed system utilizes a bidirectional cloud-based architecture designed for real-time monitoring and automated sanitation control. The ESP32-WROOM-32 microcontroller processes inputs from the ultrasonic sensor (user detection) and the water flow sensor (water consumption monitoring). The collected telemetry data are transmitted directly to the Firebase Realtime Database through Wi-Fi connectivity.

Cloud-based IoT monitoring systems have demonstrated reliable low-latency communication suitable for real-time sanitation monitoring and analytics (Satheesh Kumar et al., 2025). This architecture enables near real-time flush actuation, water usage tracking, and centralized system monitoring.

The implementation of a cloud infrastructure eliminates the need for local server maintenance while improving scalability, accessibility, and system maintainability.

3.2 System Components

3.2.1 Hardware Components

Microcontroller:

ESP32-WROOM-32 (NodeMCU-32S) was selected due to its built-in 2.4 GHz Wi-Fi capability, allowing direct communication with cloud services without additional networking modules.

Sensors:

• HC-SR04 Ultrasonic Sensor – Detects user presence to automate lid control and flushing.

• YF-S201 Water Flow Sensor – Measures water consumption in real time.

Actuators:

• Mini Submersible Water Pump (3V–5V DC) – Controls flushing mechanism.

• SG90/MG90S Servo Motors – Controls lid opening and closing.

Disinfection Component:

• UV-C LED Strip (5V DC) – Used for post-use surface disinfection. Ultraviolet-C light has been shown to effectively inactivate pathogens on surfaces (Kitagawa et al., 2021).

Power Supply Components:

• **2× Power banks** - Secondary backup power source

3.2.2 Software and Cloud Infrastructure

Frontend Technologies:
• React 18
• Next.js 14
• DaisyUI
• Recharts
• React Native (Expo or bare workflow

These technologies are used to build an administrative dashboard that displays usage statistics, water consumption trends, and device status.

Cloud Database:

Firebase Firestore is used as the cloud NoSQL database for storing sensor telemetry, event logs, user data, and system analytics. Firestore supports real-time synchronization through its onSnapshot listeners, which enables the dashboard to reflect sensor updates without requiring manual page refresh. Its document-collection data model is well-suited for the semi-structured telemetry data produced by IoT systems (Usha et al., 2025).

Authentication:

Firebase Authentication manages secure user login using email and password credentials, replacing locally managed JWT authentication mechanisms.

IoT Communication Library:

MQTT (Message Queuing Telemetry Transport) via HiveMQ Cloud serves as the communication layer between the ESP32 microcontroller and the Next.js backend server. The ESP32 uses the PubSubClient library to publish sensor telemetry and subscribe to hardware command topics over a TLS-secured connection (port 8883). The Next.js backend subscribes to these MQTT topics and writes the received data to Firebase Firestore via the Firebase Admin SDK.

3.3 Communication and Networking Architecture

The system implements bidirectional cloud communication.

3.3.1 Hardware-to-Cloud Communication

The ESP32 processes sensor data and publishes telemetry payloads to an MQTT broker (HiveMQ Cloud) over Wi-Fi using TLS encryption (port 8883). The Next.js backend server subscribes to the relevant MQTT topics and writes the received data to Firebase Firestore via the Firebase Admin SDK. Data transmitted include:

- User detection status (ultrasonic distance)
- Water flow rate and total volume per flush
- UV disinfection cycle completion status
- Device activity logs and system state transitions.

3.3.2 Cloud-to-Frontend Monitoring

The Next.js dashboard integrates the Firebase SDK with real-time listeners. When sensor values are updated in the database, the frontend automatically reflects the changes without requiring manual page refresh. This enables continuous monitoring.

In addition to the web dashboard, the system delivers real-time push notifications to the React Native mobile application installed on the maintenance personnel's device. When the system completes a UV disinfection cycle or detects an anomaly requiring manual intervention, Firebase Cloud Messaging (FCM) transmits a push notification to the registered maintenance device. The maintenance personnel can acknowledge the cleaning task and mark it as complete directly from the mobile application, with the updated status written back to Firebase Firestore and reflected on the administrative web dashboard in real time.

3.3.3 Frontend-to-Hardware Control

For manual override (e.g., manual flush), the web dashboard sends a POST request to the Next.js backend API. The backend publishes the corresponding command to the MQTT broker, which the ESP32 receives through its active subscription. The observed control latency through this pipeline is approximately 1–1.5 seconds, which is acceptable for sanitation automation systems (Satheesh Kumar et al., 2025).

3.3.4 Communication Architecture

Cloud-to-Mobile Notification Firebase Cloud Messaging serves as the notification delivery layer between the cloud backend and the maintenance personnel's mobile device. When a cleaning trigger condition is met such as the completion of a UV cycle, a hardware alert, or a manually issued cleaning request from the administrator the Next.js backend publishes a notification payload to FCM, which delivers it to the maintenance personnel's registered device. The maintenance personnel interacts with the notification through the React Native mobile application, acknowledging receipt and marking the cleaning task as completed. All task status updates are persisted to Firestore and synchronized with the administrative dashboard in real time.

Chapter 4

Hardware and Software Implementation

This chapter presents the hardware and software implementation of the Smart Flush System. It details the physical components integrated into the automated toilet fixture, the development tools and platforms used across firmware, backend, and frontend layers, and the manner in which all components are unified into a coherent end-to-end IoT pipeline.

4.1 Hardware Components (e.g., sensors, microcontrollers)

The Smart Flush System employs a set of embedded hardware components mounted on or near the toilet fixture. Each component serves a specific function within the automated cycle, detecting user presence, controlling mechanical actuation, measuring water consumption, and delivering ultraviolet sanitation.

ESP32-WROOM-32 (DOIT DevKit V1) The ESP32-WROOM-32, mounted on a DOIT DevKit V1 breakout board, serves as the central processing unit of the system. Its dual-core Xtensa LX6 processor, integrated 802.11 b/g/n Wi-Fi module, and sufficient general-purpose input/output (GPIO) pins make it well-suited for coordinating multiple sensors and actuators simultaneously. The ESP32 executes the system's finite state machine firmware, manages sensor polling at defined intervals, controls all actuators, and maintains a persistent encrypted MQTT connection to HiveMQ Cloud for real-time data transmission.

HC-SR04 Ultrasonic Distance Sensor User presence detection is handled by the HC-SR04 ultrasonic sensor. The sensor emits a trigger pulse through GPIO 13 and listens for the echo return on GPIO 12. By measuring the time-of-flight of the reflected ultrasonic wave, the firmware calculates the distance of the nearest object in centimeters. When the measured

St. Dominic Complex, E. Aguinaldo Highway, Bacoor City, Cavite, Philippines 4102
0917 625 8222 • 0922 850 9200 • (046) 417 7322 • (046) 417 8161
www.stdominiccollege.edu.ph stdominiccollege @sdcapikemen

distance falls below the configurable threshold (default: 30 cm), the system identifies a person as present and initiates the automated sequence.

SG90 Micro Servo Motors (×2) Two SG90 servo motors manage the automated opening and closing of the toilet lid. Servo 1 is connected to GPIO 25 and Servo 2 to GPIO 26. Upon person detection, both servos sweep simultaneously from 0° (closed position) to 90° (open position); following user departure, both return from 90° to 0°. Simultaneous actuation is enforced in firmware to maintain mechanical symmetry of the dual-panel lid assembly.

YF-S201 Water Flow Sensor The volume of water dispensed during each flush cycle is measured by the YF-S201 turbine-type flow sensor, connected to GPIO 19. Water flowing through the sensor spins an internal rotor, generating electrical pulses at a rate proportional to flow. The ESP32 counts these pulses via a hardware interrupt on the RISING edge, declared with the IRAM_ATTR attribute to ensure execution from fast internal memory, and converts pulse counts to liters using the sensor's calibration factor of 7.5 pulses per liter per minute. The accumulated volume is published to the MQTT broker at the conclusion of each flush event for water consumption logging and analytics.

Relay Module (Pump and UV Strip) A two-channel relay module controls the water pump and the ultraviolet sterilization strip. The pump relay, connected to GPIO 14, activates the water pump for the configured flush duration. The UV strip relay, connected to GPIO 27, activates the ultraviolet lamp for the sanitation period that follows each flush. Both relays are active-LOW, meaning a LOW signal from the ESP32 energizes the connected device, while a HIGH signal de-energizes it. This configuration ensures that a firmware fault or power interruption defaults to the safe OFF state.

The ESP32's built-in LED on GPIO 2 provides a local visual indicator of connectivity status, blinking during Wi-Fi or MQTT reconnection attempts and remaining solid when both connections are established.

4.2 Software/Platform Used

The Smart Flush System's software spans three distinct layers: embedded firmware on the ESP32 microcontroller, a cloud-based backend responsible for MQTT ingestion and database persistence, and a web-based dashboard providing real-time monitoring and remote device control.

Arduino IDE 2.x *(Justine)* The ESP32 firmware is developed in the Arduino IDE 2.x using C++. Three primary third-party libraries underpin the firmware implementation. PubSubClient provides the MQTT client functionality, enabling the ESP32 to publish sensor readings and subscribe to remote command topics. ArduinoJson handles JSON serialization and deserialization for all structured MQTT payloads. ESP32Servo enables precise PWM-based control of the SG90 servo motors on the ESP32's hardware timer channels. The firmware is architected around a non-blocking state machine using millis()-based timing throughout, ensuring that MQTT keep-alive processing and Wi-Fi maintenance are never interrupted by sensor or actuator delays.

MQTT Broker, HiveMQ Cloud *(Justine / James shared)* HiveMQ Cloud serves as the managed MQTT broker. The ESP32 establishes a TLS-encrypted connection on port 8883, ensuring that all sensor data transmitted over public networks is protected in transit. The broker routes six outbound topics published by the ESP32, covering ultrasonic distance readings, water flow measurements, and device events, and four inbound command topics to which the ESP32 subscribes for remote actuation.

Backend, Firebase and Railway *(James)* The backend infrastructure is built across two cloud services. Google Firebase Firestore serves as the primary real-time database, storing all sensor readings, event logs, device configuration documents, and usage statistics in structured collections. A standalone MQTT listener service, developed in TypeScript and deployed on Railway's free tier, maintains a persistent MQTT subscription to all ESP32 publish topics on HiveMQ Cloud. Upon receiving a message, the listener deserializes the JSON payload and

writes a structured document to the corresponding Firestore collection using the Firebase Admin SDK. Firebase Authentication provides identity management for dashboard access, restricting device data visibility to authorized users.

Frontend, Next.js 14 on Vercel *(Christian)* The monitoring and control dashboard is a Next.js 14 application built with the App Router and written in TypeScript 5. The user interface is constructed with DaisyUI 4 component library and styled using Tailwind CSS, enabling a responsive and component-driven layout accessible across desktop and mobile browsers. The application is deployed on Vercel in the Singapore region (sin1) to minimize latency for users in the Philippines. Firestore real-time listeners integrated into the dashboard's data layer push live updates to all monitoring panels, including sensor readings, flush event logs, and water volume charts, without requiring manual page refresh.

React Native Mobile Application *(Justine / Christian shared)* The maintenance personnel mobile application is developed using React Native, enabling a single codebase to target both iOS and Android platforms. Firebase Authentication restricts access to registered maintenance personnel accounts. Push notifications are delivered through Firebase Cloud Messaging, alerting maintenance staff when manual cleaning is required. The application provides four core functions: viewing the current toilet status and active alerts, acknowledging an assigned cleaning task, marking a cleaning task as complete, and reviewing the cleaning history log. All data interactions use the Firebase SDK connected to the same Firestore instance as the web dashboard, ensuring consistent real-time data across both interfaces.

4.3 **System Integration** (how the components work together)

The physical prototype enclosure used in this system is a small-scale plastic toilet model, which provides a geometrically representative environment for validating sensor placement, lid actuation mechanics, and flush mechanism integration. The use of scale models for IoT smart toilet prototype demonstration is an established practice in the literature; Zhang et al. (2021) developed their AI-Toilet IoT sensor system using a fast-prototyped scale model at the National University of Singapore, while Parkar et al. (2024) similarly employed a physical toilet model

prototype to validate smart sanitation IoT features including maintenance notification systems. This approach allows the team to verify full system behavior from ultrasonic detection through MQTT telemetry to dashboard response within a controlled and visually representative enclosure before any full-scale fixture integration.

System Data Flow (Device → Cloud → Dashboard)

The data pipeline begins at the toilet fixture. When the HC-SR04 ultrasonic sensor detects a user, measuring a distance below the configured threshold, the ESP32 firmware transitions from the STANDBY state to the PERSON_DETECTED state, triggering servo lid opening. The firmware then waits in the LID_OPEN state, monitoring the sensor for user departure. Once the user leaves and the distance exceeds the threshold for a sustained period, the system transitions through LID_CLOSING, FLUSHING (during which the pump is activated and the YF-S201 measures water volume), and UV_ACTIVE (during which the UV sterilization strip runs) states before returning to STANDBY. Throughout each transition, the ESP32 publishes structured JSON payloads to the corresponding MQTT topics on HiveMQ Cloud via an encrypted TLS connection on port 8883.

Concurrently, the backend evaluates whether the completed event meets a cleaning notification trigger condition. If triggered, the Next.js backend publishes a push notification payload to Firebase Cloud Messaging, which delivers the alert to the maintenance personnel's mobile device.

The Railway-hosted MQTT listener service receives these payloads in real time over a persistent MQTT subscription. It deserializes each payload and writes a timestamped document to the appropriate Firestore collection, for example, distance readings are written to /sensors/ultrasonic, flush events to /events/pump, and UV events to /events/uv. The Next.js dashboard on Vercel maintains active Firestore real-time listeners, which surface incoming documents as live updates to dashboard charts, usage analytics panels, and event log tables.

Command Path (Dashboard → Cloud → Device)

Remote actuation flows in the reverse direction. When a dashboard user issues a command, such as activating the pump, triggering UV sterilization, or opening the lid, the Next.js application sends the command through a server-side API route, which publishes the corresponding MQTT message to HiveMQ Cloud (e.g., toilet/commands/pump: "ON"). The ESP32, which maintains an active subscription to all four command topics, receives the message immediately and executes the corresponding actuator action. Configuration updates, such as adjusting the pump duration or detection threshold, are also delivered through this command path using the toilet/commands/config topic with a JSON payload.

Maintenance personnel interact with the system through the React Native mobile application by acknowledging assigned cleaning tasks and marking them as complete. These status updates are written to Firestore and reflected in real time on the administrative web dashboard, providing the administrator with visibility into maintenance task completion.

Chapter 5

Project Development and Implementation Process

5.1 Project Planning and Design

The development of the Klir Smart Flush System began with a structured planning phase in which the team defined the system's core objectives, identified the required hardware components, and established the overall system architecture. During this phase, the team reviewed existing smart toilet implementations and IoT sanitation solutions to identify functional gaps that the proposed system would address. The planning phase produced a component list, a system block diagram, a data flow map covering the ESP32-to-cloud-to-dashboard pipeline, and a division of responsibilities among team members across firmware, backend, and frontend layers.

St. Dominic Complex, E. Aguinaldo Highway, Bacoor City, Cavite, Philippines 4102
0917 625 8222 • 0922 850 9200 • (046) 417 7322 • (046) 417 8161
www.stdominiccollege.edu.ph stdominiccollege @sdcapikemen

The system architecture was finalized early, selecting the ESP32-WROOM-32 as the central microcontroller, HiveMQ Cloud as the MQTT broker, Firebase Firestore as the real-time database, and Next.js on Vercel as the dashboard framework, before any physical assembly began. This ensured that hardware pin assignments, MQTT topic naming conventions, and Firestore collection structures were agreed upon across all development layers prior to implementation.

5.2 Project Development Phases

The project was carried out across five sequential phases: planning and design, hardware setup, software development, system testing and validation, and final deployment and integration. Each phase was completed before the next was initiated, allowing the team to identify and resolve issues within each layer before introducing additional complexity.

5.3 Hardware Setup and Installation

Following the planning phase, the team proceeded to source and assemble all physical components. The ESP32-WROOM-32 DevKit V1 was mounted on a prototyping board and wired to the HC-SR04 ultrasonic sensor, the one SG90 servo motor, the YF-S201 water flow sensor, the four-channel relay module, and the UV-C LED strip according to the GPIO pin assignments defined during planning.

Initial hardware bring-up involved verifying each component in isolation before integrating them into the full assembly. The ultrasonic sensor was tested for accurate distance readings at known intervals, the servo motors were swept through their full range of motion, and the relay module was verified to correctly switch both the water pump and the UV strip using active-LOW logic.

One of the more demanding aspects of the hardware phase was servo motor alignment. Because the toilet lid assembly uses one SG90 servo operating simultaneously on a one-panel

mechanism, precise angular calibration was required to ensure onepanels opened and closed symmetrically without binding. The team iterated through multiple mounting positions and adjusted the firmware's PWM duty cycle values until consistent and mechanically smooth operation was achieved. Additionally, power supply stability proved to be a significant concern. Powering the ESP32, servos, pump relay, and UV strip simultaneously from a single power bank caused voltage fluctuations that intermittently reset the microcontroller. This was resolved by distributing the load across two separate power banks, one dedicated to the ESP32 and logic components, and one supplying the higher-current actuators.

The physical prototype was revised from an initial trashcan-based enclosure to a small plastic toilet model, which more accurately represents the geometry of a real toilet fixture for the purposes of sensor alignment, lid actuation demonstration, and flush mechanism validation. This revision was informed by precedent in IoT toilet research, where scale model enclosures are commonly used to demonstrate and validate embedded sensor systems prior to full-scale deployment (Zhang et al., 2021; Parkar et al., 2024). The plastic toilet model allowed the team to verify that the HC-SR04 ultrasonic sensor detection angle, servo motor lid sweep range, and relay-controlled pump outlet were physically compatible with realistic toilet geometry without requiring integration into actual plumbing infrastructure.

5.4 Software Development and Configuration

Software development proceeded in parallel across three layers once the hardware assembly was stable: ESP32 firmware, the cloud backend, and the web dashboard.

Firmware was written in C++ using Arduino IDE 2.x. The team implemented a non-blocking finite state machine using millis()-based timing to manage the system's operational states, STANDBY, PERSON_DETECTED, LID_OPEN, LID_CLOSING, FLUSHING, and UV_ACTIVE, without blocking the MQTT keep-alive loop. JSON payloads were constructed using the ArduinoJson library and published to HiveMQ Cloud over a TLS-secured MQTT connection on port 8883 using the PubSubClient library.

MQTT and Wi-Fi connectivity presented the most persistent challenge during the software phase. The ESP32 occasionally failed to reconnect to HiveMQ Cloud following a Wi-Fi dropout, causing the system to stall indefinitely. The team addressed this by implementing a reconnection loop with exponential backoff and a watchdog-style millis() timeout that forced a full connection retry cycle if neither Wi-Fi nor MQTT was re-established within a defined window. HiveMQ Cloud credentials and Wi-Fi credentials were stored as compile-time constants and the TLS fingerprint was validated to ensure connection integrity.

Backend development involved building a TypeScript-based MQTT listener service deployed on Railway. This service maintained a persistent subscription to all ESP32 publish topics and wrote incoming payloads as structured documents to Firebase Firestore using the Firebase Admin SDK. Firebase Authentication was configured to restrict dashboard access to authorized users only.

Frontend development produced the Next.js 14 dashboard deployed on Vercel in the Singapore region. The dashboard integrates Firestore real-time listeners to surface live sensor readings, flush event logs, and water consumption analytics without requiring page refresh. Manual override controls, allowing administrators to trigger flushing, UV sterilization, lid control, and configuration updates remotely, were implemented through server-side API routes that publish commands back to HiveMQ Cloud.

In parallel with frontend development, the React Native mobile application was implemented for the maintenance personnel role. The application was initialized using the Expo framework and configured with Firebase Authentication for role-restricted access. Firebase Cloud Messaging was integrated to receive push notifications from the backend. Core screens built during this phase include the notification inbox, task acknowledgment view, task completion confirmation, and cleaning history log. The application connects to the same Firestore instance as the web dashboard, ensuring that task status updates made on mobile are immediately visible to the administrator on the dashboard.

5.5 System Testing and Validation

Once hardware and software layers were individually stable, the team conducted end-to-end integration testing of the complete pipeline. Testing was performed across three areas: sensor accuracy, actuator response, and cloud communication latency.

The HC-SR04 ultrasonic sensor was evaluated across multiple detection distances to confirm consistent user presence detection at the configured 30 cm threshold. The YF-S201 water flow sensor readings were cross-referenced against a measured volume of water to validate pulse-count-to-liter conversion using the 7.5-pulse-per-liter-per-minute calibration factor. Servo motor operation was validated for symmetrical lid movement across repeated open-close cycles.

End-to-end command latency, measured from the moment a manual command was issued on the dashboard to the moment the ESP32 executed the corresponding actuator action, was observed to be approximately 1 to 1.5 seconds, consistent with the architecture's design target. Data persistence was verified by confirming that all flush events, UV cycle completions, and water volume readings were correctly written to Firestore and reflected in the dashboard in real time.

5.6 Deployment and Integration

Following successful integration testing, the system was deployed in its final configuration. The ESP32 firmware was flashed with production credentials, the Railway MQTT listener service was confirmed running continuously, and the Next.js dashboard was deployed to Vercel. The complete pipeline, from physical sensor input through cloud messaging, database persistence, and live dashboard display, was verified to operate as a unified, end-to-end system under normal operating conditions.

Chapter 6

Results and Findings

6.1 System Performance

Upon completion of integration testing, the Klir Smart Flush System was evaluated across all major subsystems: user presence detection, lid actuation, flushing and water measurement, UV-C disinfection, cloud communication latency, and dashboard monitoring. The results demonstrate that the system successfully fulfills its core design objectives, with minor limitations observed in specific hardware components.

User Presence Detection

The HC-SR04 ultrasonic sensor performed consistently throughout all test cycles, correctly identifying user presence and absence in every instance. With the detection threshold configured at 30 cm, the sensor reliably triggered the automated sequence, transitioning the firmware from STANDBY to PERSON_DETECTED, without producing false positives or missed detections during the evaluation period. The sensor's consistent performance confirms its suitability as the primary input mechanism for the system's finite state machine.

Lid Actuation

The SG90 servo motor assembly successfully automated the opening and closing of the toilet lid across all test runs. Both servos swept from 0° to 90° upon user detection and returned to 0° following user departure, as designed. However, occasional minor misalignment was observed between the one panel during repeated cycles, particularly after extended operation. This was attributed to mechanical tolerances in the lid mounting hardware rather than firmware timing errors, as the PWM signals to one servo were confirmed to be synchronized. Despite this, the lid assembly remained functional throughout all tests and did not impede normal system operation.

St. Dominic Complex, E. Aguinaldo Highway, Bacoor City, Cavite, Philippines 4102
0917 625 8222 • 0922 850 9200 • (046) 417 7322 • (046) 417 8161
www.stdominiccollege.edu.ph stdominiccollege @sdcapikemen

Flushing and Water Consumption Measurement

The flush mechanism activated reliably upon each detected user departure, with the mini submersible water pump engaging for the configured flush duration via the relay module. Water volume measurement using the YF-S201 flow sensor produced readings that were slightly offset from manually measured reference volumes; however, the margin of error was deemed acceptable for the purposes of consumption monitoring and analytics. The deviation is consistent with the known calibration characteristics of turbine-type flow sensors under variable flow rates and does not significantly affect the accuracy of cumulative water usage reporting on the dashboard.

UV-C Disinfection

The UV-C LED strip activated reliably following the conclusion of every flush cycle across all test runs. The relay controlling the UV strip engaged without failure upon the firmware's transition to the UV_ACTIVE state, and the strip completed its full disinfection period before the system returned to STANDBY. No missed or delayed UV activations were recorded during testing, indicating that the disinfection subsystem is the most consistent actuator component in the current implementation.

Cloud Communication Latency

End-to-end command latency, measured from the issuance of a manual control command on the web dashboard to the execution of the corresponding actuator action on the ESP32, was consistently observed to be approximately 1 to 1.5 seconds. This range is within the acceptable threshold for sanitation automation systems, as established in the system architecture (Satheesh Kumar et al., 2025). Sensor telemetry published by the ESP32 was received by the Railway MQTT listener and persisted to Firebase Firestore within a comparable timeframe, confirming the reliability of the upward data path.

Web Dashboard

The Next.js dashboard deployed on Vercel performed reliably throughout all monitoring sessions. Firestore real-time listeners surfaced incoming sensor readings, flush event entries, and water consumption updates to the dashboard interface instantly and without requiring manual page refresh. All monitoring panels, including the sensor status display, flush event log, and water volume analytics chart, reflected live data accurately. Remote manual override controls, including pump activation, UV trigger, and lid commands, successfully transmitted commands through the backend API route to HiveMQ Cloud and were received and executed by the ESP32 within the observed latency window.

System Performance

Mobile Notification Delivery Push notifications delivered via Firebase Cloud Messaging to the React Native mobile application were received consistently throughout all test cycles. Notification delivery latency from the moment a cleaning trigger was detected by the backend to the moment the notification appeared on the maintenance personnel's device was observed to be within an acceptable range for facility management use cases. Maintenance personnel were able to acknowledge and complete tasks directly from the mobile application, with status updates reflected on the administrative dashboard in real time.

6.2 Challenges Faced and How They Were Solved

Servo Motor Mechanical Misalignment

The servo lid assembly presented alignment challenges throughout both theB hardware setup and testing phases. Because one independent SG90 motors control a single lid structure with one panel, slight differences in motor response time and mounting angle caused occasional asymmetry during open and close cycles. The team addressed this in firmware by ensuring the servo received their PWM signals simultaneously within the same loop iteration, minimizing any software-induced timing offset. Physical mounting brackets were also adjusted iteratively to reduce mechanical play. While minor misalignment persisted under extended use, the solution achieved a functional and consistent result within the constraints of the available hardware.

St. Dominic Complex, E. Aguinaldo Highway, Bacoor City, Cavite, Philippines 4102
0917 625 8222 • 0922 850 9200 • (046) 417 7322 • (046) 417 8161
www.stdominiccollege.edu.ph stdominiccollege @sdcapikemen

MQTT and Wi-Fi Connectivity Drops

During early firmware development, the ESP32 occasionally failed to recover from Wi-Fi dropouts, causing the MQTT connection to stall and halting all telemetry transmission. This was resolved by implementing a dedicated reconnection routine using millis()-based timeouts, which continuously monitored both Wi-Fi and MQTT connection states and triggered a full reconnection cycle if either was lost for longer than a defined interval. Exponential backoff was introduced to prevent rapid repeated reconnection attempts from saturating the broker. Following this fix, the system reliably re-established connectivity after network interruptions without requiring a manual restart.

Power Supply and Wiring Instability

Power supply instability was the most impactful hardware challenge encountered during testing, directly contributing to the 20 incomplete flush cycles recorded. Powering all system components simultaneously, the ESP32, servo motor, the relay module, water pump, and UV strip, caused voltage drops during peak current draw that intermittently interrupted the flush cycle. Loose wiring connections at the relay module and pump terminals compounded this issue, occasionally causing the pump relay to disengage before the flush was complete. The team addressed this by distributing the load across two separate power banks and securing all wiring connections with more reliable terminations. While these measures improved stability, the flush success rate across the 32-cycle test period indicates that further refinement of the power distribution and wiring design remains an area for future improvement.

Challenges

Mobile Push Notification Integration Configuring Firebase Cloud Messaging for reliable push notification delivery across both iOS and Android required careful management of device registration tokens and notification payload formatting. The team addressed this by implementing a token refresh mechanism in the React Native application to ensure that FCM tokens stored in Firestore remained current, preventing missed notifications due to stale registration data.

St. Dominic Complex, E. Aguinaldo Highway, Bacoor City, Cavite, Philippines 4102
0917 625 8222 • 0922 850 9200 • (046) 417 7322 • (046) 417 8161
www.stdominiccollege.edu.ph stdominiccollege @sdcapikemen

Chapter 7

Conclusion

7.1 Summary of the Project

The Klir Smart Flush System is an ESP32-based IoT prototype designed to automate and monitor toilet hygiene through sensor-triggered lid control, automated flushing, water consumption tracking, and UV-C post-flush disinfection. The system integrates embedded hardware with a cloud-based architecture spanning HiveMQ Cloud as the MQTT broker, Firebase Firestore as the real-time database, a Railway-hosted backend listener, and a Next.js dashboard deployed on Vercel, forming a complete bidirectional IoT pipeline from physical sensor input to web-based monitoring and remote control.

The project was developed and tested by the team across five sequential phases: planning and design, hardware assembly, software development, integration testing, and deployment. A total of 32 flush cycles were conducted during the testing phase. The ultrasonic presence detection subsystem performed with complete consistency across all cycles, and the UV-C disinfection strip activated reliably following every flush event without a single failure. The web dashboard reflected live sensor data instantly and accurately throughout all monitoring sessions, and end-to-end command latency was observed at approximately 1 to 1.5 seconds, within the acceptable range for sanitation automation systems.

The primary limitation encountered was in the flushing subsystem, where only 12 out of 32 flush cycles completed successfully. Root cause analysis identified wiring instability and power supply inconsistency as the main contributing factors, with voltage drops during peak actuator current draw causing premature relay deactivation. These findings were documented honestly as areas requiring further engineering refinement in future iterations of the system.

The system additionally includes a React Native mobile application for maintenance personnel, delivering real-time push notifications when cleaning is required and enabling task acknowledgment and completion tracking from a mobile device.

Overall, the Klir Smart Flush System successfully demonstrates that an affordable, locally developed IoT solution can address real hygiene concerns in shared and public restroom environments. It solves a genuine public health problem through automated sanitation, demonstrates meaningful IoT integration in an everyday appliance, and provides administrators with data-driven insights for restroom management, all within a scalable, cloud-powered architecture.

7.2 Future Work and Enhancements (Recommendations)

While the Klir Smart Flush System achieved its core objectives as a working prototype, several areas for improvement and expansion were identified throughout the development and testing process. The following recommendations address both the limitations observed during testing and functional gaps that fall outside the current project scope.

Improved Power Distribution System

The most critical hardware improvement identified is a redesign of the power distribution architecture. The current reliance on two consumer-grade power banks introduces voltage instability under simultaneous actuator load, which was directly responsible for the majority of incomplete flush cycles during testing. Future iterations should incorporate a dedicated regulated power supply with sufficient current capacity to drive all components simultaneously, along with proper decoupling capacitors to suppress voltage spikes at the relay and motor terminals. A more reliable and permanent wiring harness with secured terminal connections would further reduce the risk of intermittent failures observed during extended testing.

Improved Lid Mechanism

The dual SG90 servo motor assembly, while functional, exhibited occasional mechanical misalignment under prolonged operation due to the physical tolerances of the mounting hardware. Future development should explore the use of higher-torque servo motors with more precise angular resolution, or an alternative lid actuation mechanism such as a linear actuator, to achieve more consistent and durable lid operation across a greater number of cycles.

St. Dominic Complex, E. Aguinaldo Highway, Bacoor City, Cavite, Philippines 4102
0917 625 8222 • 0922 850 9200 • (046) 417 7322 • (046) 417 8161
www.stdominiccollege.edu.ph stdominiccollege @sdcapikemen

More Accurate Water Flow Measurement

The YF-S201 turbine-type flow sensor provided acceptable cumulative readings for the purpose of this prototype but introduced a degree of measurement offset that could not be cross-validated against manual reference measurements during testing. Future iterations should consider replacing or supplementing the YF-S201 with a higher-precision flow meter, or implementing a field calibration routine that allows the pulse-to-liter conversion factor to be adjusted based on measured reference volumes. This would improve the reliability of water consumption analytics presented on the dashboard.

Expanded Mobile Application Capabilities

The current maintenance personnel mobile application provides core notification and task acknowledgment functionality. Future iterations could expand the mobile application to include in-app chat between maintenance staff and administrators, photo documentation of completed cleaning tasks, and offline task queuing for environments with intermittent connectivity.

Integration with Smart Home Platforms

Future versions of Klir could be integrated with established smart home ecosystems such as Google Home, Amazon Alexa, or Apple HomeKit. This would allow the system to participate in broader home or facility automation workflows, for example, triggering the toilet's UV cycle as part of a nightly cleaning routine, or surfacing water usage data alongside other utility metrics in a unified smart home dashboard. Such integration would expand the system's utility beyond standalone restroom management and position it as a component of a comprehensive connected environment.

References

Kitagawa, H., Kaiki, Y., Tadera, K., Nomura, T., Omori, K., Ikeya, S., ... (2021). Effectiveness of 222-nm ultraviolet light on disinfecting SARS-CoV-2 surface contamination. *American Journal of Infection Control, 49*(3), 299–301.

Overdrive IoT. (2024). *How IoT Can Provide Smart Restroom Solutions*. Retrieved from https://overdriveiot.com

Satheesh Kumar, G. S., Bhavani, V., Gayathri, R., & others. (2025). Cloud-based smart toilet monitoring and analytics system. *International Journal of Advanced Research in Computer Science*.

Crimaldi, J. P. (2022). CU scientists shine light on how toilet plume spreads. *University of Colorado Boulder News*. Retrieved from https://www.colorado.edu

Streamlinebath. (2025). *Role of Smart Toilets in Water Conservation*. Retrieved from https://streamlinebath.com

Firebase. (2024). *Firebase Cloud Messaging documentation*. Google. Retrieved from https://firebase.google.com/docs/cloud-messaging

Meta Platforms. (2024). *React Native documentation*. Retrieved from https://reactnative.dev/docs/getting-started

Zhang, Z., He, Q., Dong, B., Shi, Q., Zeng, X., Liu, G., & Lee, C. (2021). Artificial intelligence of toilet (AI-Toilet) for an integrated health monitoring system (IHMS) using smart triboelectric pressure sensors and image sensor. *Nano Energy, 92*, 106804. https://doi.org/10.1016/j.nanoen.2021.106804

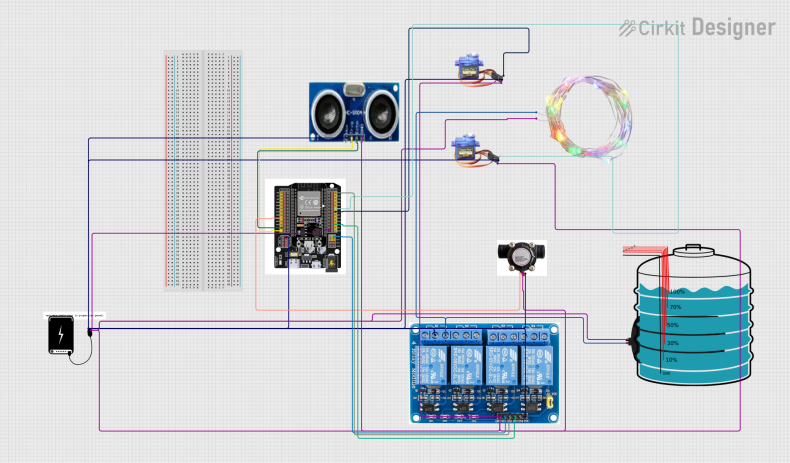
Parkar, et al. (2024). HygieniTech – The Smart Toilet. International Journal of Modern Developments in Engineering and Science, 3(1). https://journal.ijmdes.com/ijmdes/article/download/195/221/308

St. Dominic Complex, E. Aguinaldo Highway, Bacoor City, Cavite, Philippines 4102
0917 625 8222 • 0922 850 9200 • (046) 417 7322 • (046) 417 8161
www.stdominiccollege.edu.ph stdominiccollege @sdcapikemen

Raendran, V., Ramasamy, R. K., Rosdi, I. S., Razak, R., & Fauzi, N. M. (2020). IoT technology for facilities management: Understanding end user perception of the smart toilet. International Journal of Advanced Computer Science and Applications, 11(5), 353–359.

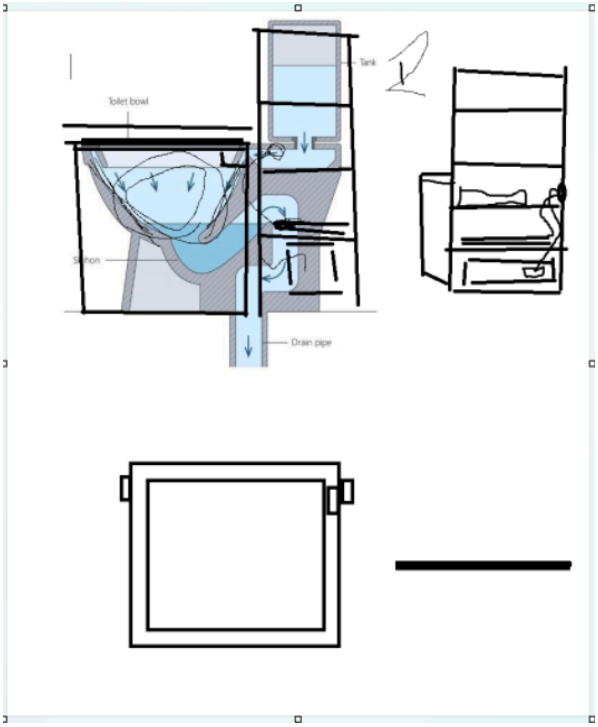
Appendices

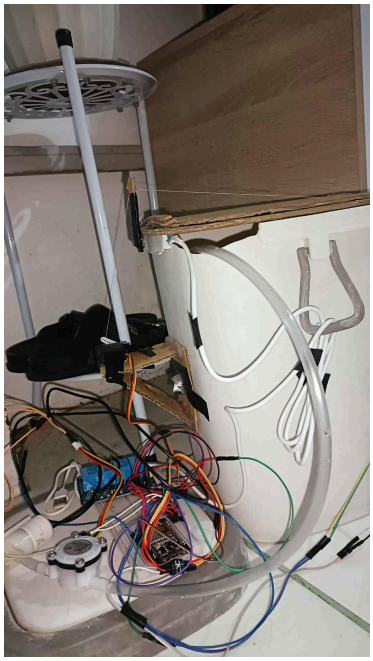
Appendix A - Diagrams, Schematics, or Screenshots





St. Dominic Complex, E. Aguinaldo Highway, Bacoor City, Cavite, Philippines 4102
0917 625 8222 • 0922 850 9200 • (046) 417 7322 • (046) 417 8161
www.stdominiccollege.edu.ph stdominiccollege @sdcapikemen





St. Dominic Complex, E. Aguinaldo Highway, Bacoor City, Cavite, Philippines 4102
0917 625 8222 • 0922 850 9200 • (046) 417 7322 • (046) 417 8161
www.stdominiccollege.edu.ph stdominiccollege @sdcapikemen

Appendix B - Code or Scripts Used

```
// ── 1. Includes + Defines ──────────────────────────────────────
#include <WiFi.h>
#include <WiFiClientSecure.h>
#include <PubSubClient.h>
#include <ArduinoJson.h>
#include <ESP32Servo.h>

// ── 2. Credentials ─────────────────────────────────────────────
#define WIFI_SSID
#define WIFI_PASSWORD
#define MQTT_BROKER
#define MQTT_PORT
#define MQTT_USER
#define MQTT_PASS

// ── 3. Pin Definitions ─────────────────────────────────────────
#define TRIG_PIN        13
#define ECHO_PIN        12
#define PUMP_PIN        14
#define UV_PIN          27
#define SERVO1_PIN      25
#define SERVO2_PIN      26
#define FLOW_PIN        32
#define LED_PIN         2

// ── 4. Config Parameters ───────────────────────────────────────
int DETECTION_THRESHOLD_CM = 30;
int PUMP_DURATION_MS       = 3000;
int UV_DURATION_MS         = 5000;
int PERSON_GONE_CONFIRM_MS = 3000;
```

```
// ── 5. State Enum ──────────────────────────────────────────
enum State {
  STANDBY,
  PERSON_DETECTED,
  LID_OPEN,
  WAITING_FOR_DEPARTURE,
  LID_CLOSING,
  FLUSHING,
  UV_ACTIVE
};

State currentState = STANDBY;

// ── 6. Global Variables ────────────────────────────────────
Servo servo1;
Servo servo2;

volatile int pulseCount  = 0;
float totalVolume        = 0;
float flushDuration      = 0;

unsigned long lastUltrasonicPublish = 0;
unsigned long lastDistanceTrigger   = 0;
unsigned long lastReconnectAttempt  = 0;
unsigned long lastLedBlink          = 0;
unsigned long personGoneTimer       = 0;
unsigned long pumpStartTime         = 0;
unsigned long uvStartTime           = 0;
unsigned long flushStartTime        = 0;

float distanceBuffer[5] = {0, 0, 0, 0, 0};
```

```
int distanceIndex       = 0;
bool ledState           = false;

WiFiClientSecure espClient;
PubSubClient client(espClient);

// ── 7. Flow Sensor ISR ──────────────────────────────────────
void IRAM_ATTR pulseCounter() {
  pulseCount++;
}

// ── 8. WiFi ─────────────────────────────────────────────────
void connectWiFi() {
  Serial.printf("[%lu] [WIFI] Connecting to %s...\n", millis(),
WIFI_SSID);
  WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
  unsigned long start = millis();
  while (WiFi.status() != WL_CONNECTED) {
    if (millis() - start >= 20000) {
      Serial.printf("[%lu] [WIFI] Timeout!\n", millis());
      return;
    }
    delay(500);
    Serial.print(".");
  }
  Serial.printf("\n[%lu] [WIFI] Connected! IP: %s\n", millis(),
    WiFi.localIP().toString().c_str());
}

// ── 9. MQTT Callback ────────────────────────────────────────
```

```
void mqttCallback(char* topic, byte* payload, unsigned int
length) {
  String message = "";
  for (int i = 0; i < length; i++) message += (char)payload[i];
  Serial.printf("[%lu] [MQTT] Received [%s]: %s\n", millis(),
topic, message.c_str());

  if (String(topic) == "toilet/commands/pump") {
    digitalWrite(PUMP_PIN, message == "ON" ? LOW : HIGH);
    Serial.printf("[%lu] [PUMP] Manual %s\n", millis(),
message.c_str());
  }

  if (String(topic) == "toilet/commands/uv") {
    digitalWrite(UV_PIN, message == "ON" ? LOW : HIGH);
    Serial.printf("[%lu] [UV] Manual %s\n", millis(),
message.c_str());
  }

  if (String(topic) == "toilet/commands/lid") {
    if (message == "OPEN")  openLid();
    if (message == "CLOSE") closeLid();
  }

  if (String(topic) == "toilet/commands/config") {
    StaticJsonDocument<200> doc;
    deserializeJson(doc, message);
    if (doc.containsKey("pumpDuration"))
      PUMP_DURATION_MS = (int)doc["pumpDuration"] * 1000;
    if (doc.containsKey("uvDuration"))
      UV_DURATION_MS = (int)doc["uvDuration"] * 1000;
    if (doc.containsKey("threshold"))
      DETECTION_THRESHOLD_CM = (int)doc["threshold"];
```

```
    Serial.printf("[%lu] [CONFIG] pump:%dms uv:%dms
threshold:%dcm\n",
      millis(), PUMP_DURATION_MS, UV_DURATION_MS,
DETECTION_THRESHOLD_CM);
  }
}

// ── 10. MQTT Connect
──────────────────────────────────────────

bool connectMQTT() {
  Serial.printf("[%lu] [MQTT] Connecting...\n", millis());
  if (client.connect("ESP32SmartFlush", MQTT_USER, MQTT_PASS)) {
    Serial.printf("[%lu] [MQTT] Connected!\n", millis());
    client.subscribe("toilet/commands/pump");
    client.subscribe("toilet/commands/uv");
    client.subscribe("toilet/commands/lid");
    client.subscribe("toilet/commands/config");
    digitalWrite(LED_PIN, HIGH);
    return true;
  }
  Serial.printf("[%lu] [MQTT] Failed rc=%d\n", millis(),
client.state());
  return false;
}

void reconnectMQTT() {
  if (!client.connected() && millis() - lastReconnectAttempt >=
5000) {
    lastReconnectAttempt = millis();
    connectMQTT();
  }
}
```

```
// ── 11. Servo Functions (continuous rotation, time-based) ──────
#define SPEED_OPEN   0      // full speed one direction
#define SPEED_CLOSE  180    // full speed opposite direction
#define OPEN_TIME    1600   // milliseconds to fully open
#define CLOSE_TIME   1300   // milliseconds to fully close
#define STOP_POS     92     // calibrated neutral stop

void openLid() {
  Serial.printf("[%lu] [LID] Opening...\n", millis());
  servo1.attach(SERVO1_PIN);
  servo2.attach(SERVO2_PIN);
  delay(100);               // allow attach to settle

  // Rotate both servos at full speed in the open direction
  servo1.write(SPEED_OPEN);
  servo2.write(SPEED_OPEN);
  delay(OPEN_TIME);

  // Stop both servos
  servo1.write(STOP_POS);
  servo2.write(STOP_POS);
  delay(500);               // extra time to stop completely

  servo1.detach();
  servo2.detach();

  // Publish lid open event
  StaticJsonDocument<100> doc;
  doc["status"]    = "open";
  doc["timestamp"] = millis();
  char buffer[100];
  serializeJson(doc, buffer);
  client.publish("toilet/events/lid", buffer);
```

St. Dominic Complex, E. Aguinaldo Highway, Bacoor City, Cavite, Philippines 4102
0917 625 8222 • 0922 850 9200 • (046) 417 7322 • (046) 417 8161
www.stdominiccollege.edu.ph stdominiccollege @sdcapikemen


```
  Serial.printf("[%lu] [LID] Open — event published\n",
millis());
}

void closeLid() {
  Serial.printf("[%lu] [LID] Closing...\n", millis());
  servo1.attach(SERVO1_PIN);
  servo2.attach(SERVO2_PIN);
  delay(100);

  // Rotate both servos at full speed in the close direction
  servo1.write(SPEED_CLOSE);
  servo2.write(SPEED_CLOSE);
  delay(CLOSE_TIME);

  // Stop both servos
  servo1.write(STOP_POS);
  servo2.write(STOP_POS);
  delay(500);

  servo1.detach();
  servo2.detach();

  // Publish lid closed event
  StaticJsonDocument<100> doc;
  doc["status"]    = "closed";
  doc["timestamp"] = millis();
  char buffer[100];
  serializeJson(doc, buffer);
  client.publish("toilet/events/lid", buffer);
  Serial.printf("[%lu] [LID] Closed — event published\n",
millis());
}
```

```
// ── 12. Distance Sensor ──────────────────────────────
float getDistance() {
  if (millis() - lastDistanceTrigger >= 200) {
    lastDistanceTrigger = millis();
    digitalWrite(TRIG_PIN, LOW);
    delayMicroseconds(2);
    digitalWrite(TRIG_PIN, HIGH);
    delayMicroseconds(10);
    digitalWrite(TRIG_PIN, LOW);
    long duration = pulseIn(ECHO_PIN, HIGH, 60000);
    float dist = duration / 58.0;
    if (dist > 0 && dist < 400) {
      distanceBuffer[distanceIndex % 5] = dist;
      distanceIndex++;
    }
  }
  // Median of last 5
  float sorted[5];
  memcpy(sorted, distanceBuffer, sizeof(sorted));
  for (int i = 0; i < 4; i++)
    for (int j = i + 1; j < 5; j++)
      if (sorted[i] > sorted[j]) {
        float tmp = sorted[i];
        sorted[i] = sorted[j];
        sorted[j] = tmp;
      }
  return sorted[2];
}

void publishDistance(float distance) {
  if (millis() - lastUltrasonicPublish >= 1000) {
```

```
    lastUltrasonicPublish = millis();
    StaticJsonDocument<100> doc;
    doc["distance"]  = distance;
    doc["unit"]      = "cm";
    doc["timestamp"] = millis();
    char buffer[100];
    serializeJson(doc, buffer);
    client.publish("toilet/sensors/ultrasonic", buffer);
    Serial.printf("[%lu] [SENSOR] Distance: %.1f cm\n",
millis(), distance);
  }
}

// ── 13. LED Status ──────────────────────────────────────

void updateLED() {
  if (WiFi.status() != WL_CONNECTED) {
    if (millis() - lastLedBlink >= 200) {
      ledState = !ledState;
      digitalWrite(LED_PIN, ledState);
      lastLedBlink = millis();
    }
  } else if (!client.connected()) {
    if (millis() - lastLedBlink >= 1000) {
      ledState = !ledState;
      digitalWrite(LED_PIN, ledState);
      lastLedBlink = millis();
    }
  } else {
    digitalWrite(LED_PIN, HIGH);
  }
}
```

```
// ── 14. State Machine ──────────────────────────────────────
void updateStateMachine(float distance) {
  switch (currentState) {

    case STANDBY:
      if (distance > 0 && distance < DETECTION_THRESHOLD_CM) {
        currentState = PERSON_DETECTED;
        Serial.printf("[%lu] [STATE] STANDBY → 
PERSON_DETECTED\n", millis());
      }
      break;

    case PERSON_DETECTED:
      // Open lid completely before moving to next state
      openLid();
      currentState    = LID_OPEN;
      personGoneTimer = 0;
      Serial.printf("[%lu] [STATE] PERSON_DETECTED → 
LID_OPEN\n", millis());
      break;

    case LID_OPEN:
      // Lid is fully open — just wait for person to leave
      currentState = WAITING_FOR_DEPARTURE;
      Serial.printf("[%lu] [STATE] LID_OPEN → 
WAITING_FOR_DEPARTURE\n", millis());
      break;

    case WAITING_FOR_DEPARTURE:
      if (distance >= DETECTION_THRESHOLD_CM) {
        if (personGoneTimer == 0) {
          personGoneTimer = millis();
```

```
          Serial.printf("[%lu] [STATE] Person may have left —
confirming...\n", millis());
        } else if (millis() - personGoneTimer >=
PERSON_GONE_CONFIRM_MS) {
          personGoneTimer = 0;
          currentState    = LID_CLOSING;
          Serial.printf("[%lu] [STATE] WAITING_FOR_DEPARTURE →
LID_CLOSING\n", millis());
        }
      } else {
        if (personGoneTimer != 0) {
          personGoneTimer = 0;
          Serial.printf("[%lu] [STATE] Person still present —
resetting timer\n", millis());
        }
      }
      break;

    case LID_CLOSING:
      // Close lid COMPLETELY before pump starts
      closeLid();
      delay(500); // small pause after lid fully closed

      // Now start flush
      pulseCount     = 0;
      totalVolume    = 0;
      flushStartTime = millis();
      pumpStartTime  = millis();
      digitalWrite(PUMP_PIN, LOW);

      // Publish pump active
      {
        StaticJsonDocument<100> doc;
```

```
        doc["status"]    = "active";
        doc["timestamp"] = millis();
        char buffer[100];
        serializeJson(doc, buffer);
        client.publish("toilet/events/pump", buffer);
      }

      currentState = FLUSHING;
      Serial.printf("[%lu] [STATE] LID_CLOSING → FLUSHING\n",
millis());
      Serial.printf("[%lu] [PUMP] ON\n", millis());
      break;

    case FLUSHING:
      if (millis() - pumpStartTime >= PUMP_DURATION_MS) {
        digitalWrite(PUMP_PIN, HIGH);
        flushDuration = (millis() - flushStartTime) / 1000.0;

        // Calculate volume
        noInterrupts();
        int pulses = pulseCount;
        pulseCount = 0;
        interrupts();
        float flowRate = (pulses / 7.5);
        totalVolume   += (flowRate / 60.0);

        // Publish waterflow
        {
          StaticJsonDocument<100> doc;
          doc["volume"]    = totalVolume;
          doc["duration"]  = flushDuration;
          doc["unit"]      = "L";
          char buffer[100];
```

```
        serializeJson(doc, buffer);
        client.publish("toilet/sensors/waterflow", buffer);
      }

      // Publish pump inactive
      {
        StaticJsonDocument<100> doc;
        doc["status"]    = "inactive";
        doc["timestamp"] = millis();
        char buffer[100];
        serializeJson(doc, buffer);
        client.publish("toilet/events/pump", buffer);
      }

      Serial.printf("[%lu] [PUMP] OFF — %.2f L in %.1f s\n",
        millis(), totalVolume, flushDuration);

      // Start UV
      digitalWrite(UV_PIN, LOW);
      uvStartTime  = millis();
      currentState = UV_ACTIVE;
      Serial.printf("[%lu] [STATE] FLUSHING → UV_ACTIVE\n",
millis());
      Serial.printf("[%lu] [UV] ON\n", millis());
    }
    break;

  case UV_ACTIVE:
    if (millis() - uvStartTime >= UV_DURATION_MS) {
      digitalWrite(UV_PIN, HIGH);

      // Publish UV cycle
      {
```

```
            StaticJsonDocument<100> doc;
            doc["duration"]  = UV_DURATION_MS / 1000;
            doc["completed"] = true;
            doc["timestamp"] = millis();
            char buffer[100];
            serializeJson(doc, buffer);
            client.publish("toilet/events/uv", buffer);
          }

          Serial.printf("[%lu] [UV] OFF — cycle complete\n",
millis());
          currentState = STANDBY;
          Serial.printf("[%lu] [STATE] UV_ACTIVE → STANDBY\n",
millis());
          Serial.printf("[%lu] [SYSTEM] Ready for next person!\n",
millis());
        }
        break;
    }
}

// ── 15. Setup ──────────────────────────────────────────────
void setup() {
  Serial.begin(115200);

  pinMode(TRIG_PIN, OUTPUT);
  pinMode(ECHO_PIN, INPUT);
  pinMode(PUMP_PIN, OUTPUT);
  pinMode(UV_PIN,   OUTPUT);
  pinMode(LED_PIN,  OUTPUT);

  digitalWrite(PUMP_PIN, HIGH);
```

```
  digitalWrite(UV_PIN,    HIGH);
  digitalWrite(LED_PIN,   LOW);

  servo1.detach();
  servo2.detach();

  pinMode(FLOW_PIN, INPUT_PULLUP);
  attachInterrupt(digitalPinToInterrupt(FLOW_PIN), pulseCounter,
RISING);

  espClient.setInsecure();
  connectWiFi();
  client.setServer(MQTT_BROKER, MQTT_PORT);
  client.setCallback(mqttCallback);
  connectMQTT();

  Serial.printf("[%lu] [SYSTEM] Smart Flush ready — STANDBY\n",
millis());
}

// ── 16. Loop
──────────────────────────────────────────────────

void loop() {
  if (WiFi.status() != WL_CONNECTED) {
    Serial.printf("[%lu] [WIFI] Dropped — reconnecting...\n",
millis());
    connectWiFi();
  }

  if (!client.connected()) reconnectMQTT();
  client.loop();

  float distance = getDistance();
```

St. Dominic Complex, E. Aguinaldo Highway, Bacoor City, Cavite, Philippines 4102
0917 625 8222 • 0922 850 9200 • (046) 417 7322 • (046) 417 8161
www.stdominiccollege.edu.ph stdominiccollege @sdcapikemen


```
  if (client.connected()) publishDistance(distance);

  updateStateMachine(distance);

  updateLED();
}
```

Appendix C - Authors

James Carl Alvarez,
Dennis Carlos,
Maria Lindog,
Justine Lopez,
Christian Los Baños

St. Dominic Complex, E. Aguinaldo Highway, Bacoor City, Cavite, Philippines 4102
0917 625 8222 • 0922 850 9200 • (046) 417 7322 • (046) 417 8161
www.stdominiccollege.edu.ph stdominiccollege @sdcapikemen